INTRODUCTION TO DEEP LEARNING

Course number: 00240332

Lecture 6: Training techniques

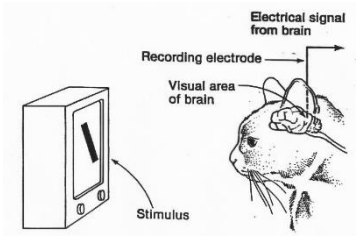
Xiaolin Hu (胡晓林)

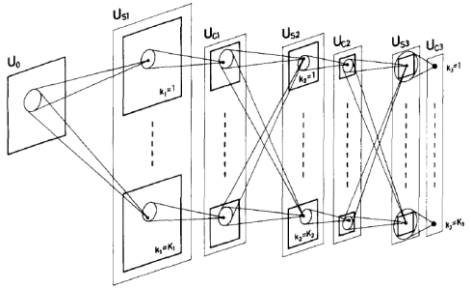
Dept. of Computer Science and Technology

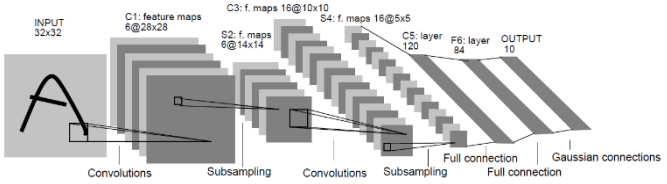
Tsinghua University

Last lecture review

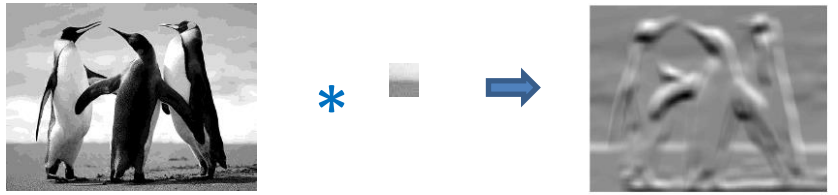
I. Introduction







II. Convolutional layer



III. Pooling layer



Outline

I. Optimizers
II. Techniques for dealing with overfitting
III. Batch normalization
IV. Summary

Recap: SGD and momentum

- SGD optimizes over individual minibatches $\left(\boldsymbol{X}^{(i)}, \boldsymbol{t}^{(i)}\right)$ at each iteration

$$\boldsymbol{g} = \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}\right)$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \boldsymbol{g}$$

- The momentum update is given by,

$$\boldsymbol{v} = \gamma \boldsymbol{v} - \eta \boldsymbol{g}$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} + \boldsymbol{v}$$

or

$$\boldsymbol{v} = \gamma \boldsymbol{v} + \eta \boldsymbol{g}$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \boldsymbol{v}$$

- We need to adjust learning rates η during training
  - Recall different strategies

The method manipulates the learning rate globally and equally for all parameters

Discussion

Is it the best strategy to set the same learning rate for all parameters?

Can we use different learning rates for different parameters? And how?



Per-parameter adaptive learning rate

Adagrad

- An adaptive learning rate method (Duchi et al. 2011)
- The updating rule

$$\boldsymbol{c} = \boldsymbol{c} + \boldsymbol{g}^2$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

$\boldsymbol{g} = \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}\right)$ is the gradient

Elementwise operations

where ϵ is usually set between 10^{-4} and 10^{-8}

$\boldsymbol{c}$ is used to normalize the parameter update step

Suppose at a certain iteration, $c_3 = 90, c_9 = 2$, then

$$\Delta\theta_3 = -\frac{\eta}{\sqrt{90 + \epsilon}} g_3 \qquad \Delta\theta_9 = -\frac{\eta}{\sqrt{2 + \epsilon}} g_9$$

Effective learning rate

Parameters received small updates will have larger effective learning rates

Adagrad

- An adaptive learning rate method (Duchi et al. 2011)
- Denote the gradient by $\boldsymbol{g} = \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}\right)$
- The updating rule

$$\boldsymbol{c} = \boldsymbol{c} + \boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

Elementwise operations

  where ϵ is usually set between 10^{-4} and 10^{-8}
- Any problem with this method?

Adagrad

- An adaptive learning rate method (Duchi et al. 2011)
- Denote the gradient by $\boldsymbol{g} = \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}\right)$
- The updating rule

$$\boldsymbol{c} = \boldsymbol{c} + \boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

Elementwise operations

  where ϵ is usually set between 10^{-4} and 10^{-8}
- Any problem with this method?
  - The effective learning rates are monotonically decreasing, which may leads to early stopping

Why?

RMSProp

- Adagrad: let $\boldsymbol{c}$ accumulate $\boldsymbol{g}^2$ at all previous steps
- RMSProp: let $\boldsymbol{c}$ accumulate the recent $\boldsymbol{g}^2$
- In practice

$$\boldsymbol{c} = \gamma \boldsymbol{c} + (1 - \gamma)\boldsymbol{g}^2$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

- Typical values for γ are 0.9, 0.99, 0.999
- RMSProp still modulates the learning rate of each parameter based on the magnitudes of its gradients, but unlike Adagrad the updates do not get monotonically smaller

Adagrad versus RMSProp

Adagrad: $\boldsymbol{c}(t) = \boldsymbol{c}(t-1) + \boldsymbol{g}(t)^2$

$= \boldsymbol{c}(0) + \boldsymbol{g}(1)^2 + \boldsymbol{g}(2)^2 + \ldots + \boldsymbol{g}(t)^2$

All $\boldsymbol{g}(1)^2, \ldots, \boldsymbol{g}(t)^2$ contribute equally to $\boldsymbol{c}(t)$

RMSProp: $\boldsymbol{c}(t) = \gamma \boldsymbol{c}(t-1) + (1-\gamma)\boldsymbol{g}(t)^2$

$= \gamma^t \boldsymbol{c}(0) + \gamma^{t-1}(1-\gamma)\boldsymbol{g}(1)^2 + \gamma^{t-2}(1-\gamma)\boldsymbol{g}(2)^2 + \ldots$

$+ \gamma(1-\gamma)\boldsymbol{g}(t-1)^2 + (1-\gamma)\boldsymbol{g}(t)^2$

Product of t numbers between (0,1)

Product of $t-1$ numbers between (0,1)

Contributions of $\boldsymbol{g}(k)^2$ to $\boldsymbol{c}(t)$ far away from t decay exponentially

How to introduce momentum?

Let $\boldsymbol{g} = \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}\right)$

SGD

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \boldsymbol{g}$$

→ **SGD+momentum**

$$\boldsymbol{v} = \gamma \boldsymbol{v} + \eta \boldsymbol{g}$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \boldsymbol{v}$$

RMSProp

$$\boldsymbol{c} = \gamma \boldsymbol{c} + (1 - \gamma)\boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

→

$$\boldsymbol{v} = \beta \boldsymbol{v} + \eta \boldsymbol{g}$$
$$\boldsymbol{c} = \gamma \boldsymbol{c} + (1 - \gamma)\boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \frac{\boldsymbol{v}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

Adam

$$\boldsymbol{m} = \beta_1 \boldsymbol{m} + (1 - \beta_1)\boldsymbol{g}$$
$$\boldsymbol{c} = \beta_2 \boldsymbol{c} + (1 - \beta_2)\boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{m}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

Adam

$$\boldsymbol{m} = \beta_1 \boldsymbol{m} + (1 - \beta_1)\boldsymbol{g}$$
$$\boldsymbol{c} = \beta_2 \boldsymbol{c} + (1 - \beta_2)\boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{m}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

Recommended values: $\epsilon = 10^{-8}, \beta_1 = 0.9, \beta_2 = 0.999$

- Proposed by Kingma and Ba (2015). Default algorithm!
- The full version ("warm up" version)

At the beginning, $\boldsymbol{m}$ is small

$$\boldsymbol{m} = \beta_1 \boldsymbol{m} + (1 - \beta_1)\boldsymbol{g}, \qquad \boldsymbol{m}_t = \boldsymbol{m}/(1 - \beta_1^t)$$
$$\boldsymbol{c} = \beta_2 \boldsymbol{c} + (1 - \beta_2)\boldsymbol{g}^2, \qquad \boldsymbol{c}_t = \boldsymbol{c}/(1 - \beta_2^t)$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{m}_t}{\sqrt{\boldsymbol{c}_t + \epsilon}}$$

where t denotes the iteration

Summary of Part I

Let $\boldsymbol{g} = \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}\right)$

SGD

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \boldsymbol{g}$$

SGD+momentum

$$\boldsymbol{v} = \gamma \boldsymbol{v} - \eta \boldsymbol{g}$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} + \boldsymbol{v}$$

Adagrad

$$\boldsymbol{c} = \boldsymbol{c} + \boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

RMSProp

$$\boldsymbol{c} = \gamma \boldsymbol{c} + (1 - \gamma)\boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

Adam

$$\boldsymbol{m} = \beta_1 \boldsymbol{m} + (1 - \beta_1)\boldsymbol{g}$$
$$\boldsymbol{c} = \beta_2 \boldsymbol{c} + (1 - \beta_2)\boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{m}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

- Which is the best?
- All optimizers have a learning rate η
  - Learning rate decay is always a good strategy

More details about optimization techniques

http://cs231n.github.io/neural-networks-3

多选题 1分

Setting

Which updating rules allow different learning rates for different parameters?

A Stochastic gradient decent (SGD)

B SGD+momentum

C Adagrad

D RMSProp

E Adam

Submit

单选题 1分

Setting

Which updating rule has the early stopping problem?

A Adagrad

B RMSprop

C Adam

Submit

单选题 1分

设置

Does Adam algorithm follow the exact gradient of the loss function?

$$\boldsymbol{m} = \beta_1 \boldsymbol{m} + (1 - \beta_1)\boldsymbol{g}$$
$$\boldsymbol{c} = \beta_2 \boldsymbol{c} + (1 - \beta_2)\boldsymbol{g}^2$$
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{m}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

A Yes

B No

提交

Outline

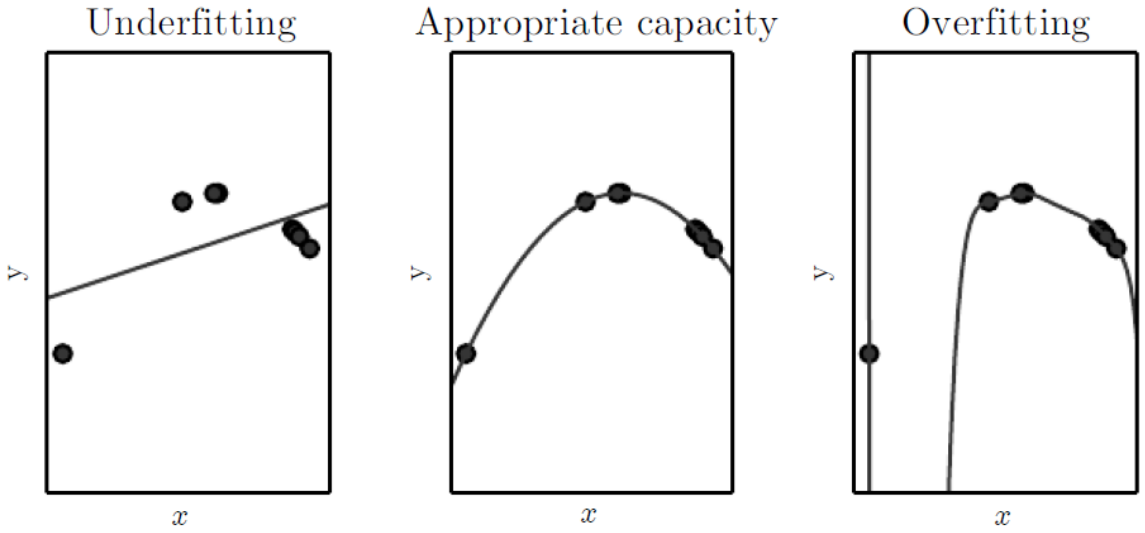
I. Optimizers
II. Techniques for dealing with overfitting
III. Batch normalization
IV. Summary

Recall: polynomial regression example

- Consider a regression problem in which the input x and output y are both scalars. Find a function $f: \mathbb{R} \to \mathbb{R}$ to fit the data
  - $f(x) = b + wx$
  - $f(x) = b + w_1 x + w_2 x^2$
  - $f(x) = b + \sum_{i=1}^{9} w_i x^i$

MSE training:

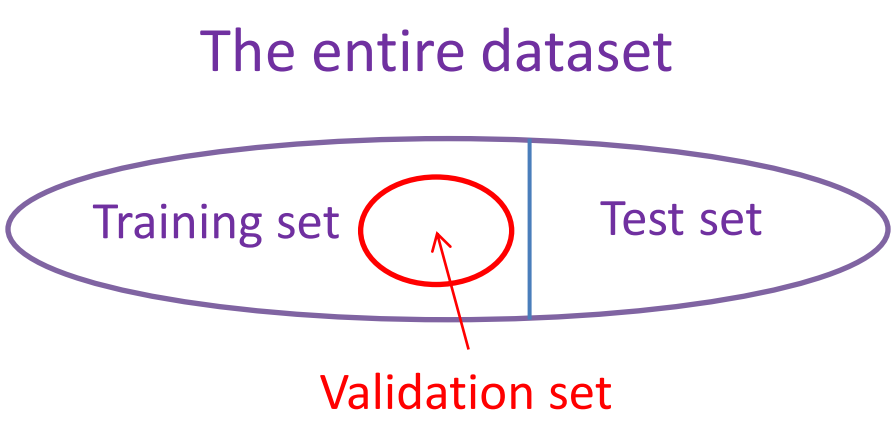
$$\min_w \frac{1}{N} \sum_{n=1}^{N} \left|\left| f\left(x^{(n)}\right) - y^{(n)} \right|\right|_2^2$$

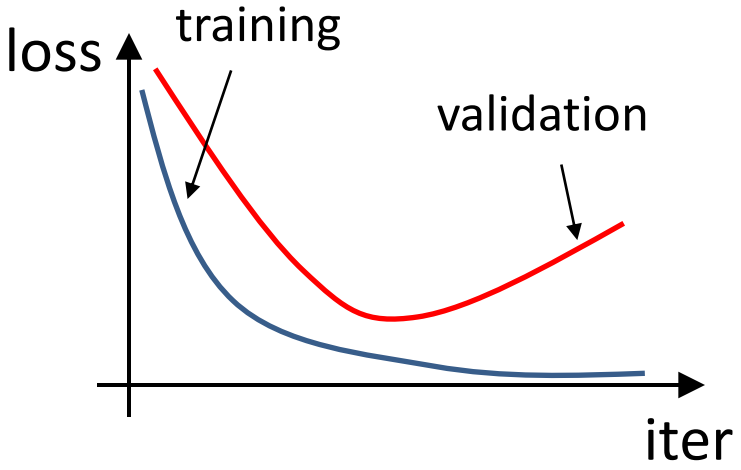


Overfitting

- A neural network (as well as other machine learning models) typically contains many parameters to learn (e.g., millions to billions) which tends to overfit the training data
- What's overfitting?
  - Fits the training data well but performs poorly on held-out test data
- Weight regularization is one method to handle this
- Other techniques
  - Early stopping
  - Dropout (Hinton et al., 2012)
  - Data augmentation

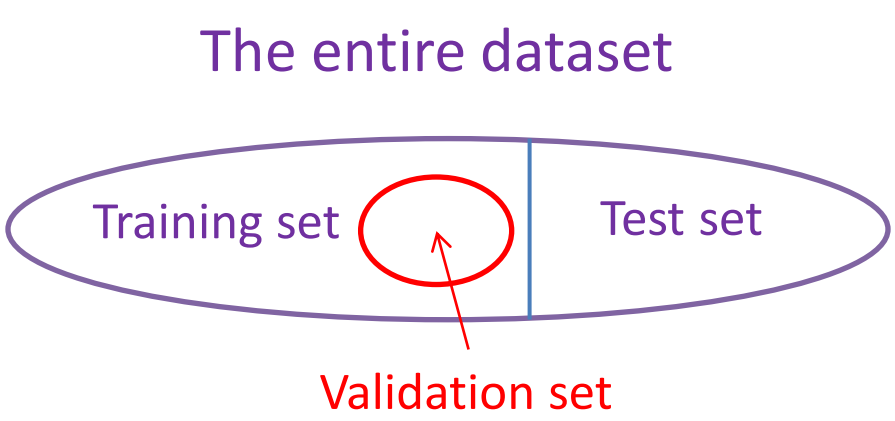
Early stopping

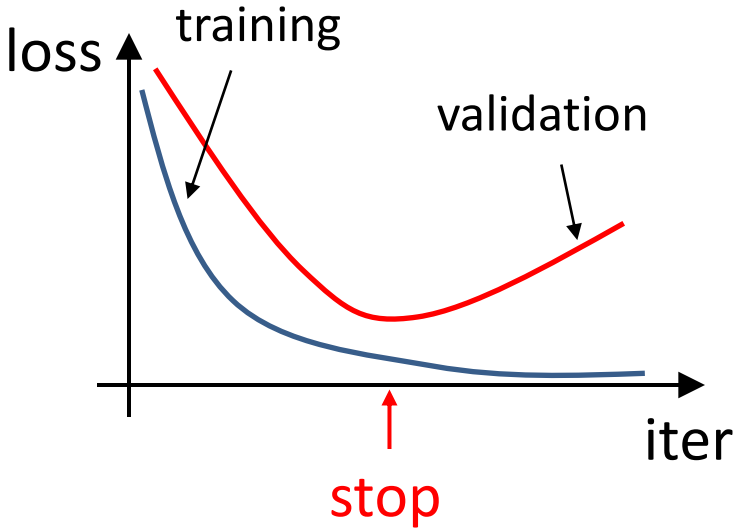




- When you see the loss on the training set is decreasing, but the loss on the validation set begins to increase…

Early stopping

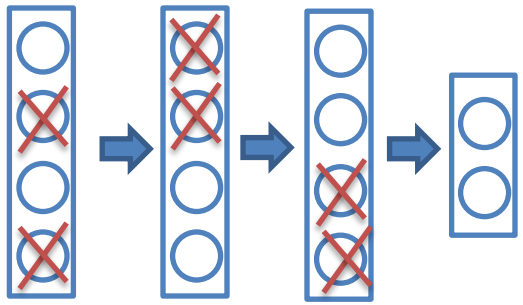




- When you see the loss on the training set is decreasing, but the loss on the validation set begins to increase…STOP there!

Dropout

- On each presentation of each training case, each hidden unit is randomly omitted (zero its output) from the network with a probability p (e.g., 0.5)

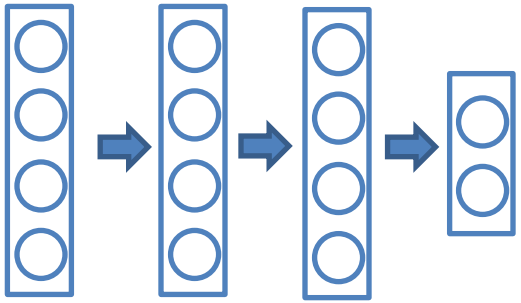


These zero values are used in the backward pass propagating the derivatives to the parameters

- Advantage
  - A hidden unit cannot rely on other hidden units being present, therefore we prevent complex co-adaptations of the neurons on the training data
  - It trains a huge number of different networks in a reasonable time, then average their predictions

Testing phase

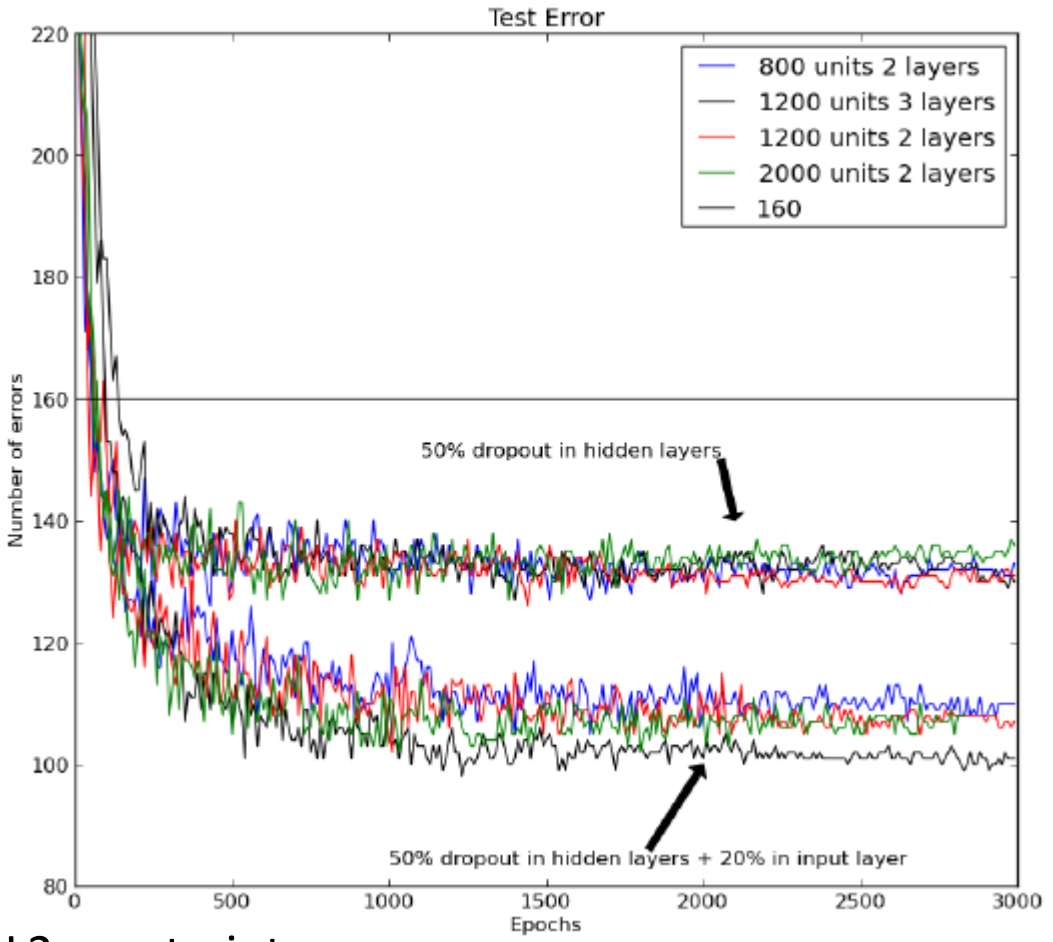
- Use the "mean network" that contains all of the hidden units



- But we need to adjust the outgoing weights of neurons to compensate for the fact that in training only a portion of them are active
  - If $p = 0.5$, we halve the weights
  - If $p = 0.1$, we multiply the weights with $1 - p$, i.e., 0.9
- In practice, this gives very similar performance to averaging over a large number of dropout networks

Results on MNIST

- Standard MLP without the tricks
  - Enhancing training data with transformed images
  - Generative pre-training
- In this setting without dropout the best results is 160 errors
- Dropout can significantly reduce errors



Another trick is used: separate L2 constraints on the incoming weights of each hidden unit

Hinton et al., 2012

Remarks

- In some implementations, during test, $(1-p)$ is multiplied with the output of the activation function, say $f(Wx+b)$, instead of the weights W. Then

$$a_{\text{test}} = (1-p)f(Wx+b)$$

- In some implementations, the output of the activation function is changed as follows

$$a_{\text{train}} = \frac{1}{(1-p)} f(Wx+b)$$

Inverted dropout

during training, while the output of the activation function is unchanged during test

- In practice, p is set lower in lower layers, e.g., 0.2, but higher in higher layers, e.g., 0.5
- In the literature or some software, the dropout rate is sometimes defined as the probability for retaining the output of each node, i.e., $(1-p)$

Discussion

- Inspired by dropout, can you figure out other techniques to alleviate overfitting?
  - Dropconnect by Wan, Zeiler, Zhang, LeCun, Fergus (2013)
  - Drop pixels: a data augmentation method

Data augmentation

Let $\{\boldsymbol{x}^{(n)}, \boldsymbol{t}^{(n)}\}$ denote the original training set

① Add variations to the input data

$$\boldsymbol{x}^{(n)} \rightarrow \hat{\boldsymbol{x}}^{(n)}$$

while keep the label $\boldsymbol{t}^{(n)}$ unchanged

② Use the augmented training set

$$\{\boldsymbol{x}^{(n)}, \boldsymbol{t}^{(n)}\} \cup \{\hat{\boldsymbol{x}}^{(n)}, \boldsymbol{t}^{(n)}\}$$

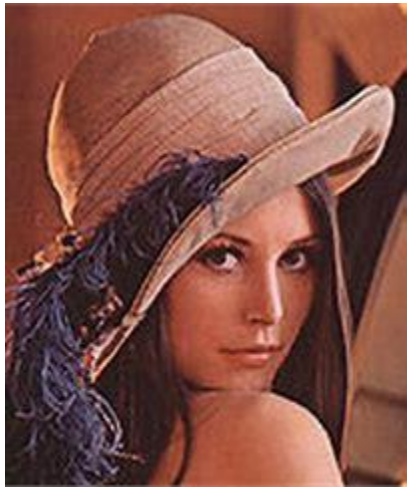
to train the model

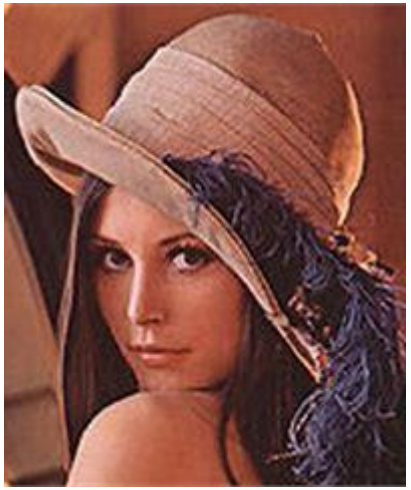
Different data (text, image, video) type can use different variations

Commonly used variations for images

- Random
  - flips
  - translations
  - crops and scales
  - stretching
  - shearing
  - Cutout or erasing
  - mixup
  - color jittering
  - etc.
- A combination of above

Random flip





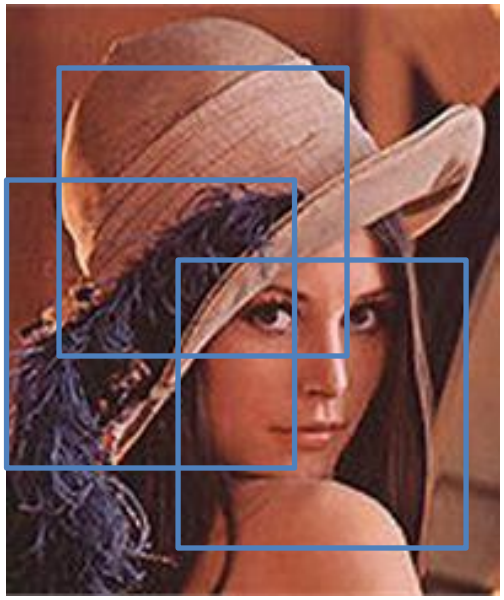
Random crops and scales

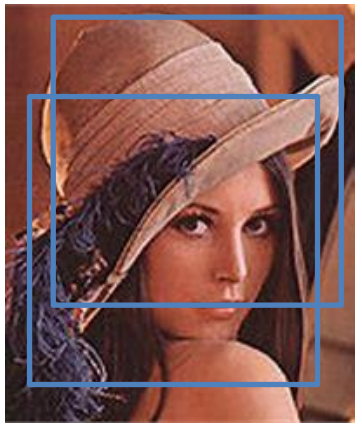
During training, you can…

crops



Scale then crop





What would you do for test?

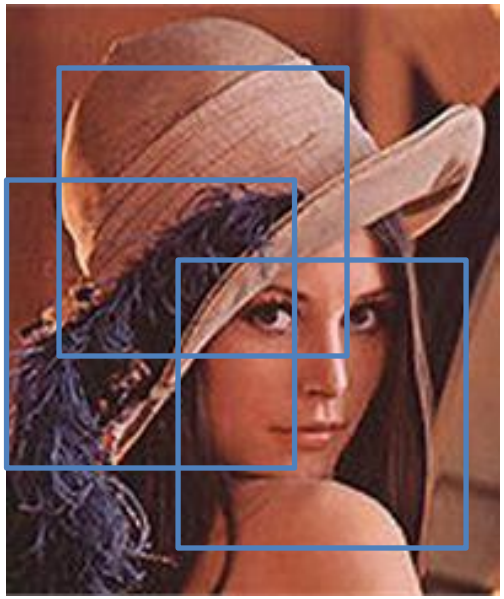
Random crops and scales

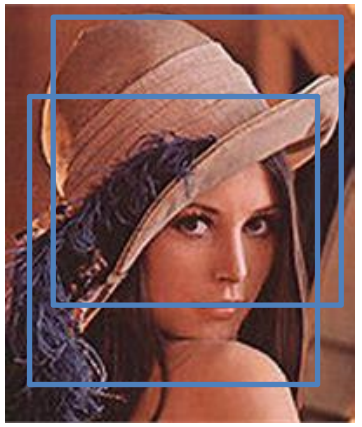
During training, you can…

crops



Scale then crop



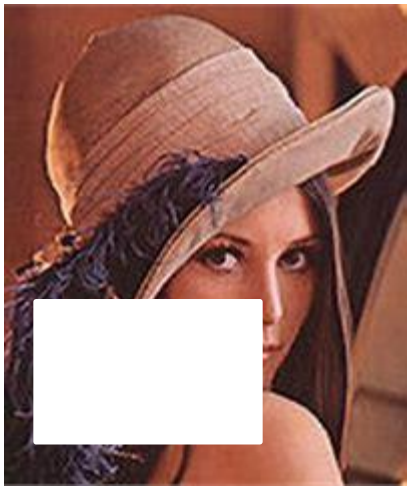


During test, you can

- Resize the test images to the required input size of the model
- Crop a region (usually the center one) and input to the model
- Crop multiple regions and input to the model, then average the predictions of the model

Random cutout or erasing

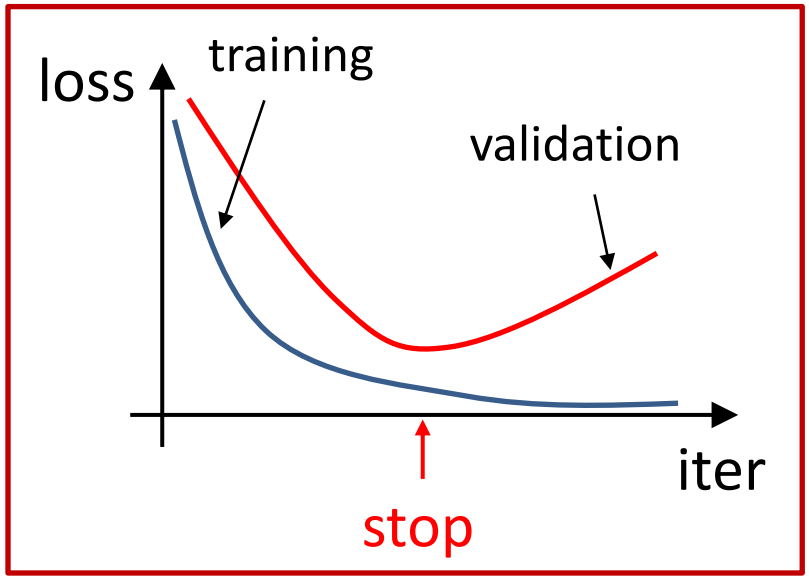
During training, you can…

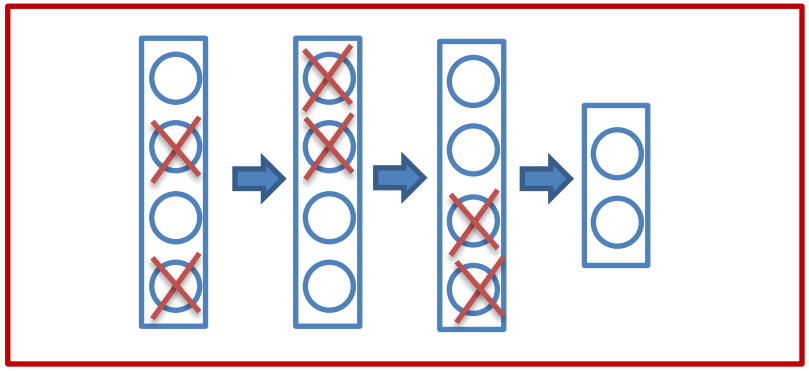

During test, you input the whole image to the model

Summary of Part II

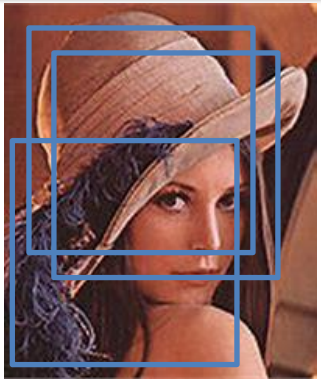
Early stop



Dropout



Data augmentation



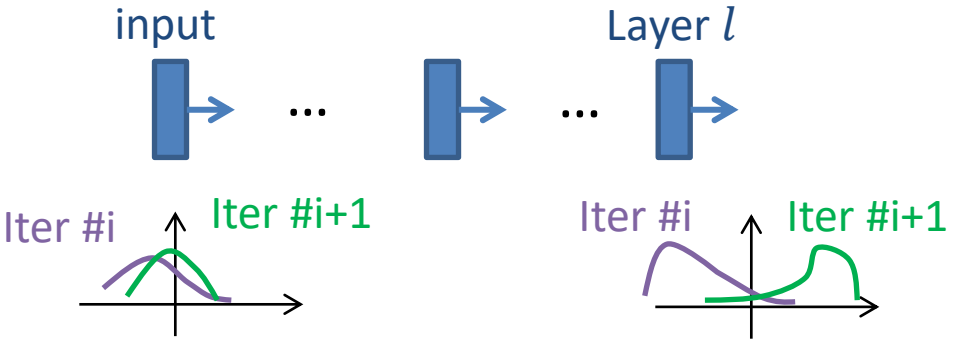
Outline

I. Optimizers
II. Techniques for dealing with overfitting
III. Batch normalization
IV. Summary

“Internal covariate shift”

- Since we use SGD, the input mini-batches to the neural network at different iterations are different
- This *may* cause the distributions of the output of a layer to be different at different iterations

Distribution of a neuron (or some neurons) over a mini-batch



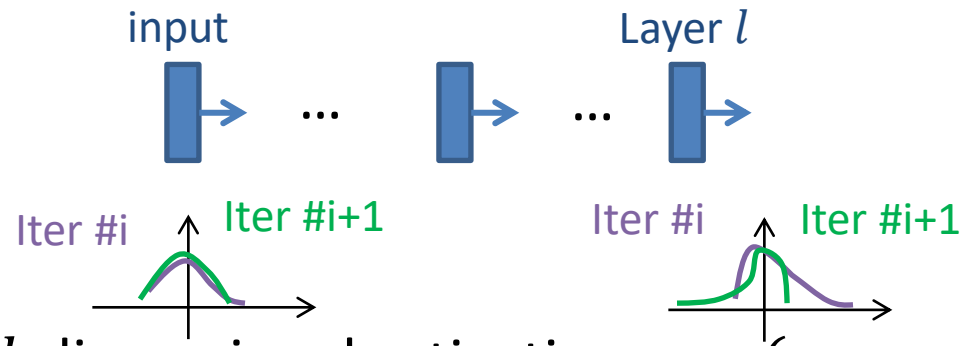
Internal Covariate Shift (ICS): The change in the distributions of internal nodes of a deep network, in the course of training

It *may* cause difficulty in optimization

Reduce ICS by normalization

Ioffe, Szegedy, 2015

- We normalize each scalar feature independently, by making it have the mean of zero and the variance of 1



- Denote a d-dimensional activation $\boldsymbol{x} = (x_1, \ldots, x_d)$, do normalization

$$\hat{x}_i = \frac{x_i - \mathrm{E}[x_i]}{\sqrt{\mathrm{Var}[x_i]}}$$

- Keep the representation ability of the layer

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

If $\gamma_i = \sqrt{\mathrm{Var}[x_i]}$ and $\beta_i = \mathrm{E}[x_i]$, then we recover the original activations

Batch normalization

- Construct a new layer:

$$\boldsymbol{y}^{(n)} = BN_{\boldsymbol{\gamma},\boldsymbol{\beta}}\left(\boldsymbol{x}^{(n)}\right)$$

- Where $\boldsymbol{\gamma}, \boldsymbol{\beta}, \boldsymbol{x}^{(n)}, \boldsymbol{y}^{(n)} \in R^d$
- Forward pass
  - $\boldsymbol{\mu}_B = \frac{1}{m}\sum_{n=1}^{M} \boldsymbol{x}^{(n)}$
  - $\boldsymbol{\sigma}_B^2 = \frac{1}{m}\sum_{n=1}^{M}\left(\boldsymbol{x}^{(n)} - \boldsymbol{\mu}_B\right)^2$
  - $\widehat{\boldsymbol{x}}^{(n)} = \frac{\boldsymbol{x}^{(n)} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \epsilon}}$
  - $\boldsymbol{y}^{(n)} = \boldsymbol{\gamma}\widehat{\boldsymbol{x}}^{(n)} + \boldsymbol{\beta}$

The arithmetic operations are elementwise

- What are needed to be computed in the backward pass?

During inference

- The normalization of activations that depends on the mini-batch allows efficient training, but is neither necessary nor desirable during inference
- Once the network has been trained, we normalize

$$\hat{\boldsymbol{x}} = \frac{\boldsymbol{x} - \mathrm{E}[\boldsymbol{x}]}{\sqrt{\mathrm{Var}[\boldsymbol{x}] + \epsilon}}$$

  where $\mathrm{E}[\boldsymbol{x}]$ and $\mathrm{Var}[\boldsymbol{x}]$ are measured over the entire training set
  - $\mathrm{E}[\boldsymbol{x}] = \mathrm{E}_B[\boldsymbol{\mu}_B]$
  - $\mathrm{Var}[\boldsymbol{x}] = \frac{m}{m-1}\mathrm{E}_B[\boldsymbol{\sigma}_B^2]$ where m is the size of the mini-batches
- If you want to track the accuracy of a model as it trains, you can use the moving averages instead

Location in a network

- Often applied before the non-linearity of the previous layer
  - Empirical results
- The previous layer is always a linear transformation layer (fully-connected layer or convolutional layer)

$$\boldsymbol{y}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)} + \boldsymbol{b}^{(l)}$$

  - The bias term can be ignored because BN has a shift term that have the same effect. Therefore

$$\boldsymbol{y}^{(l)} = \mathrm{BN}\left(\boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)}\right)$$

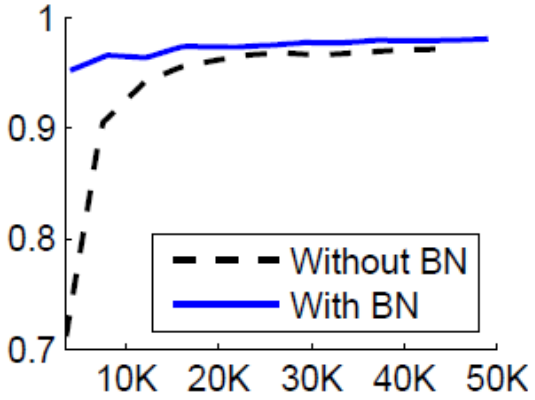
BN in CNN

- BN is applied after the convolutional layer
- Require different elements of the same feature map are normalized in the same way
- Normalize all activations in a mini-batch, over all locations
  - Suppose the mini-batch size is M, and the feature map size is $P \times Q$, then the mean and variance are calculated across $M \cdot P \cdot Q$ elements
  - Learn a pair of parameters γ_i and β_i per feature map, rather than per activation
- The inference procedure is modified similarly, so that during inference BN applies the same linear transformation to each activation in a given feature map

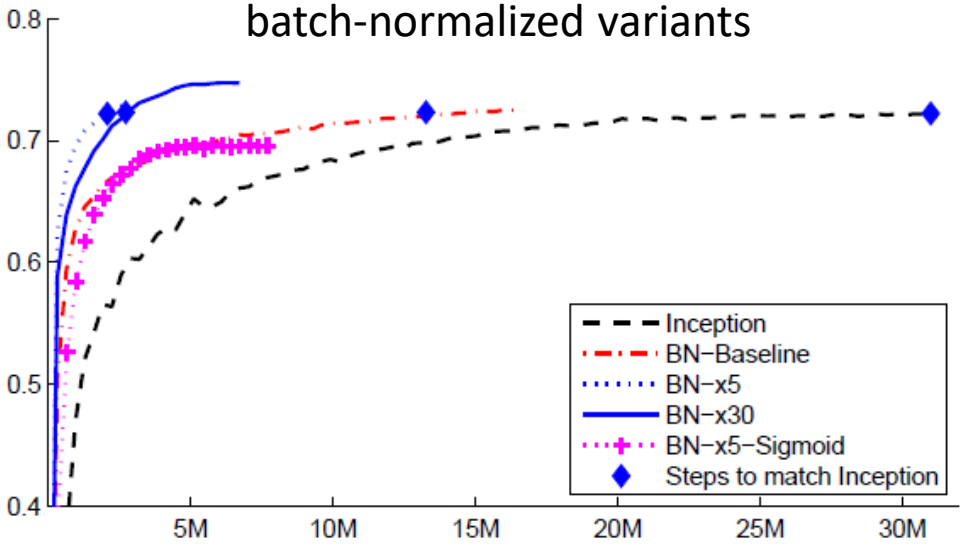
Advantages

- BN enables higher learning rates
- BN regularizes the model
  - The training network no longer produce deterministic values for a given training example
  - Dropout may not be needed

Test accuracy on the MNIST

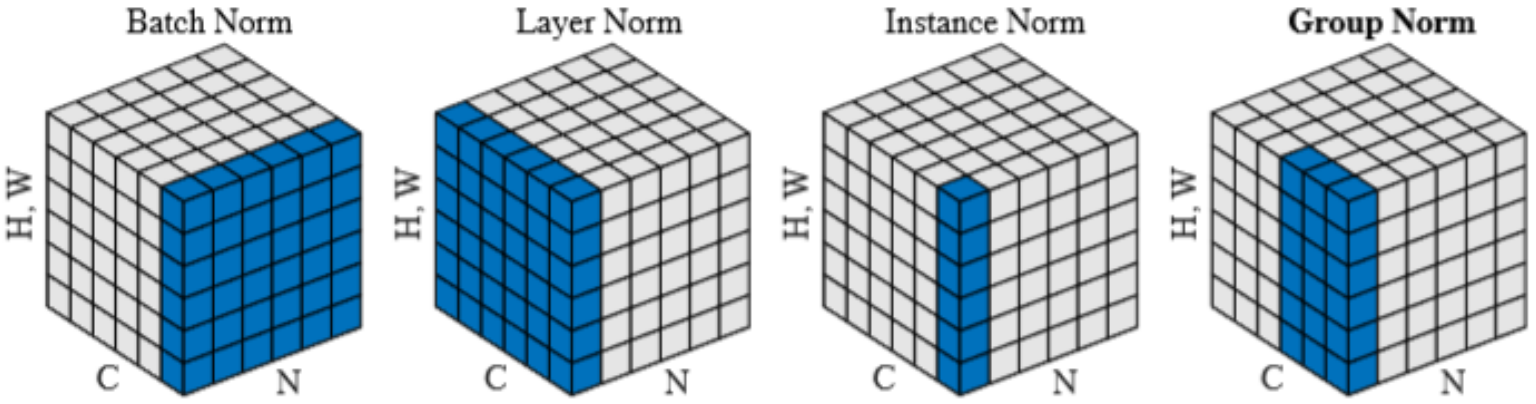


Single crop validation accuracy of Inception and its batch-normalized variants



X-axis: The number of training steps

Other normalization techniques



Wu, He, arXiv:1803.08494v3

All of them share the same linear transformation form, but do normalization over different dimensions

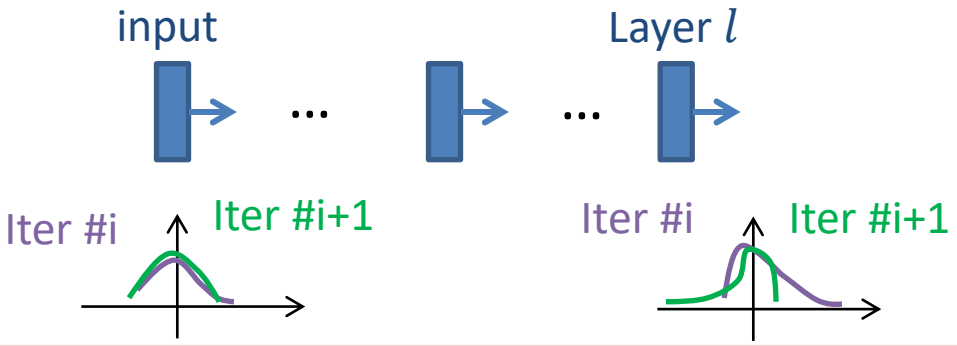
- Batch norm is usually used in CNN
- Layer norm is usually used in RNN
- Instance norm is usually used in image stylization
- Group norm is used in CNN to deal with small batchsize

More discussion: https://www.cnblogs.com/LXP-Never/p/11566064.html

Summary of Part III

Motivation

Reduce "Internal covariate shift"



Method

$$\hat{x}_i = \frac{x_i - \mathrm{E}[x_i]}{\sqrt{\mathrm{Var}[x_i]}} \qquad y_i = \gamma_i \hat{x}_i + \beta_i$$

The reason of the good performance is controversial!

单选题 1分

设置

During training, if each hidden unit is randomly omitted with a probability p. Then during test, which value should be multiplied with the weights?

A p

B 1-p

C p(1-p)

D 1/(1-p)

提交

Outline

I. Optimizers
II. Techniques for dealing with overfitting
III. Batch normalization
IV. Summary

Summary of this lecture

Knowledge

I. Optimizers

Adagrad

$$\boldsymbol{c} = \boldsymbol{c} + \boldsymbol{g}^2$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

RMSProp

$$\boldsymbol{c} = \gamma \boldsymbol{c} + (1 - \gamma)\boldsymbol{g}^2$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

Adam

$$\boldsymbol{m} = \beta_1 \boldsymbol{m} + (1 - \beta_1)\boldsymbol{g}$$

$$\boldsymbol{c} = \beta_2 \boldsymbol{c} + (1 - \beta_2)\boldsymbol{g}^2$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{m}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

II. Techniques for dealing with overfitting

Early stop | Dropout | Data augmentation

III. Batch normalization

$$\hat{x}_i = \frac{x_i - \mathrm{E}[x_i]}{\sqrt{\mathrm{Var}[x_i]}} \qquad y_i = \gamma_i \hat{x}_i + \beta_i$$

Recommended reading

- Various optimization techniques for deep learning
  http://cs231n.github.io/neural-networks-3/
- Various regularization techniques
  https://www.cnblogs.com/LXP-Never/p/11566064.html

- How to choose hyperparameters
  See handout in WebLearning

Prepare for the next lecture

- Form groups of 2-3 and every group prepares a 5-minute presentation with slides for one of the following papers
  - Santurkar et al. “How Does Batch Normalization Help Optimization? ” NeurIPS 2018
  - Li, Chen, Hu, Yang, “Understanding the Disharmony Between Dropout and Batch Normalization by Variance Shift,” CVPR 2019

Homework 2

- Pytorch basics
  - Deadline: Nov 3 (Monday noon) 12:00